UNIVERSIDAD NACIONAL DE COLOMBIA

Sede Medellín —

Programación Orientada a Objetos

Informe Técnico — Actividad 2

ESTUDIANTE
Kevin Santiago Montoya Chavarría

DOCENTE
Walter Hugo Arboleda Mazo

ASIGNATURA
Programación Orientada a Objetos

LENGUAJE DE PROGRAMACIÓN
Python 3.x

REPOSITORIO DE GITHUB DEL PROYECTO

https://github.com/Kevin007ch/POO-Kevin-Santiago-Montoya-Chavarria

Resumen de la Actividad

El presente documento consolida la solución desarrollada para los ejercicios del módulo 2 sobre fundamentos del paradigma Orientado a Objetos. Se aborda el diseño de clases, encapsulamiento, uso de enumeraciones, constructores y definición de métodos con especificación de tipos.

- **Ejercicio 2.1 — Persona:** Modelado básico de atributos de identidad y método de impresión.
- **Ejercicio 2.2 — Planeta:** Manejo de enumeraciones para tipos de cuerpo celeste y cálculo de propiedades físicas.
- **Ejercicio 2.3 — Automóvil:** Control de estado, aceleración, frenado y especificaciones técnicas.
- **Ejercicio 2.4 — Figuras Geométricas:** Modelado geométrico con cálculo de áreas, perímetros y clasificaciones.
- **Ejercicio 2.5 — Cuenta Bancaria:** Lógica transaccional con control de saldos y operaciones de consignación/retiro.

Ejercicio 2.1: Persona

Código Fuente en Python

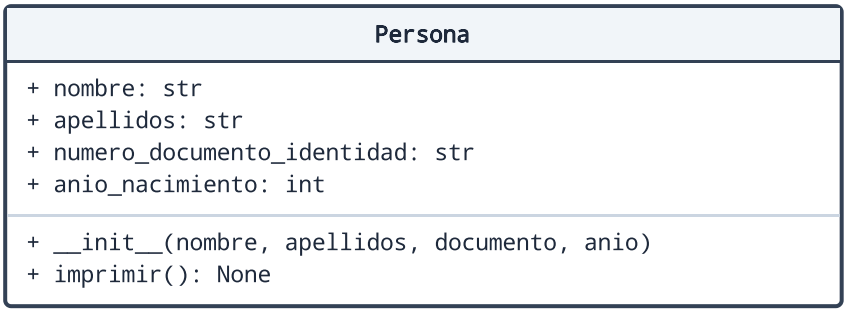
```python
class Persona:
    def __init__(self, nombre: str, apellidos: str,
                 numero_documento_identidad: str, anio_nacimiento: int):
        self.nombre = nombre
        self.apellidos = apellidos
        self.numero_documento_identidad = numero_documento_identidad
        self.anio_nacimiento = anio_nacimiento

    def imprimir(self) -> None:
        print(f"Nombre = {self.nombre}")
        print(f"Apellidos = {self.apellidos}")
        print(f"Número de documento de identidad = {self.numero_documento_identidad}")
        print(f"Año de nacimiento = {self.anio_nacimiento}")
        print()

if __name__ == "__main__":
    persona1 = Persona("Pedro", "Pérez", "1053121010", 1998)
    persona2 = Persona("Luis", "León", "1053223344", 2001)

    persona1.imprimir()
    persona2.imprimir()
```

Diagrama de Clases UML



Ejercicio 2.2: Planeta

Código Fuente en Python

```python
from enum import Enum

class TipoPlaneta(Enum):
    GASEOSO = "GASEOSO"
    TERRESTRE = "TERRESTRE"
    ENANO = "ENANO"

class Planeta:
    UA_KM = 149_597_870

    def __init__(self, nombre: str, cantidad_satelites: int, masa: float,
                 volumen: float, diametro: int, distancia_sol: int,
                 tipo: TipoPlaneta, es_observable: bool):
        self.nombre = nombre
        self.cantidad_satelites = cantidad_satelites
        self.masa = masa
        self.volumen = volumen
        self.diametro = diametro
        self.distancia_sol = distancia_sol
        self.tipo = tipo
        self.es_observable = es_observable

    def imprimir(self) -> None:
        print(f"Nombre = {self.nombre}\nSatélites = {self.cantidad_satelites}")
        print(f"Masa = {self.masa} kg | Volumen = {self.volumen} km³")

    def calcular_densidad(self) -> float:
        return self.masa / self.volumen

    def es_planeta_exterior(self) -> bool:
        return self.distancia_sol > (self.UA_KM * 3.4)
```

Diagrama UML



Ejercicio 2.3: Automóvil

Getters, Setters y Métodos Dinámicos

Código Fuente en Python

```python
from enum import Enum

class TipoCombustible(Enum):
    GASOLINA = "GASOLINA"; BIOETANOL = "BIOETANOL"; DIESEL = "DIESEL"

class TipoAutomovil(Enum):
    CARRO_DE_CIUDAD = "CARRO DE CIUDAD"; SUBCOMPACTO = "SUBCOMPACTO"; SUV = "SUV"

class Color(Enum):
    BLANCO = "BLANCO"; NEGRO = "NEGRO"; ROJO = "ROJO"; AZUL = "AZUL"

class Automovil:
    def __init__(self, marca: str, modelo: int, motor: float,
                 tipo_combustible: TipoCombustible, tipo_automovil: TipoAutomovil,
                 numero_puertas: int, cantidad_asientos: int, velocidad_maxima: int, color: Color):
        self.marca = marca
        self.modelo = modelo
        self.motor = motor
        self.tipo_combustible = tipo_combustible
        self.tipo_automovil = tipo_automovil
        self.numero_puertas = numero_puertas
        self.cantidad_asientos = cantidad_asientos
        self.velocidad_maxima = velocidad_maxima
        self.color = color
        self.velocidad_actual = 0

    def acelerar(self, incremento: int) -> None:
        if self.velocidad_actual + incremento <= self.velocidad_maxima:
            self.velocidad_actual += incremento
        else:
            print("Excede velocidad máxima.")

    def desacelerar(self, decremento: int) -> None:
        if self.velocidad_actual - decremento >= 0:
            self.velocidad_actual -= decremento

    def frenar(self) -> None:
        self.velocidad_actual = 0
```

Estructura de Clases

Automovil
- marca: str | modelo: int | motor: float - velocidad_maxima: int | velocidad_actual: int
+ acelerar(incremento: int): None + desacelerar(decremento: int): None + frenar(): None + calcular_tiempo_llegada(distancia: float): float

Ejercicio 2.4: Figuras Geométricas

Código Fuente en Python

```python
import math

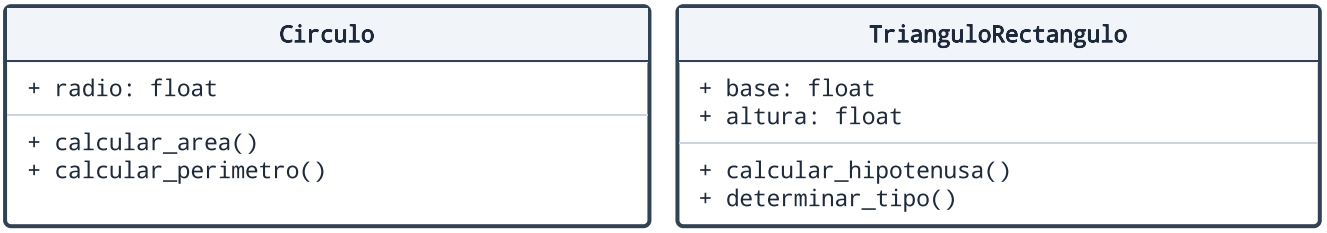
class Circulo:
    def __init__(self, radio: float): self.radio = radio
    def calcular_area(self) -> float: return math.pi * self.radio ** 2
    def calcular_perimetro(self) -> float: return 2 * math.pi * self.radio

class Rectangulo:
    def __init__(self, base: float, altura: float):
        self.base = base; self.altura = altura
    def calcular_area(self) -> float: return self.base * self.altura
    def calcular_perimetro(self) -> float: return 2 * (self.base + self.altura)

class Cuadrado:
    def __init__(self, lado: float): self.lado = lado
    def calcular_area(self) -> float: return self.lado ** 2
    def calcular_perimetro(self) -> float: return 4 * self.lado

class TrianguloRectangulo:
    def __init__(self, base: float, altura: float):
        self.base = base; self.altura = altura
    def calcular_hipotenusa(self) -> float:
        return math.sqrt(self.base**2 + self.altura**2)
    def determinar_tipo(self) -> str:
        hip = self.calcular_hipotenusa()
        if math.isclose(self.base, self.altura): return "Isósceles"
        return "Escaleno"
```

Diagrama de Clases Geometricas



Ejercicio 2.5: Cuenta Bancaria

Código Fuente en Python

```python
from enum import Enum

class TipoCuenta(Enum):
    AHORROS = "AHORROS"
    CORRIENTE = "CORRIENTE"

class CuentaBancaria:
    def __init__(self, nombres_titular: str, apellidos_titular: str,
                 numero_cuenta: int, tipo_cuenta: TipoCuenta):
        self.nombres_titular = nombres_titular
        self.apellidos_titular = apellidos_titular
        self.numero_cuenta = numero_cuenta
        self.tipo_cuenta = tipo_cuenta
        self.saldo = 0.0

    def consignar(self, valor: float) -> bool:
        if valor > 0:
            self.saldo += valor
            print(f"Consignado ${valor:,.2f}. Nuevo saldo: ${self.saldo:,.2f}")
            return True
        return False

    def retirar(self, valor: float) -> bool:
        if 0 < valor <= self.saldo:
            self.saldo -= valor
            print(f"Retirado ${valor:,.2f}. Nuevo saldo: ${self.saldo:,.2f}")
            return True
        return False
```

Diagrama UML

